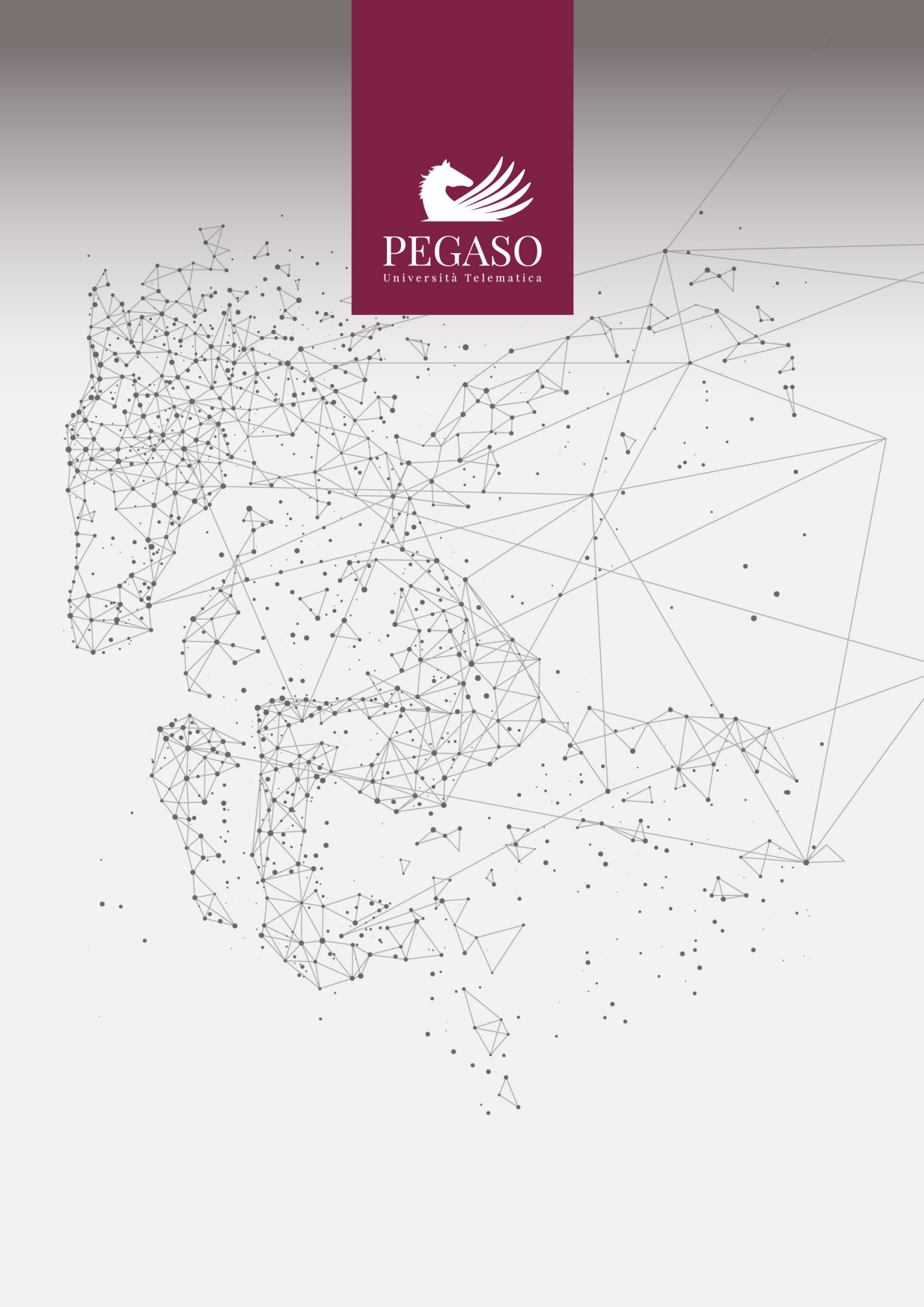

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ALGORITMO DI MONTECARLO 3
2. ESPRESSIONE POLINOMIALE 6
3. BLOOM FILTER 8
BIBLIOGRAFIA 15

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Algoritmo di Montecarlo

Nel campo della programmazione, esistono numerose strategie utilizzate per risolvere problemi e ottenere risultati in modo efficiente. Tra queste strategie, si distinguono gli **algoritmi deterministici** e gli **algoritmi probabilistici**.

Gli algoritmi deterministici sono basati su una serie di passi logici che producono una soluzione esatta e deterministica al problema in questione. Questi algoritmi sono utilizzati quando la soluzione esatta del problema può essere ottenuta con certezza e senza ambiguità. Gli algoritmi deterministici sono spesso utilizzati per risolvere problemi di ottimizzazione, ricerca e ordinamento.

D'altra parte, gli algoritmi probabilistici producono soluzioni approssimate al problema in questione, basandosi sulla probabilità invece che sulla certezza. Questi algoritmi sono utilizzati quando la soluzione esatta del problema è difficile o impossibile da ottenere con certezza, ma una soluzione approssimata può essere sufficiente per le esigenze dell'applicazione. Gli algoritmi probabilistici sono spesso utilizzati per la simulazione di fenomeni fisici, la ricerca di informazioni su Internet e la criptografia.

In generale, un algoritmo probabilistico è composto da due parti:

- la fase di preparazione
- la fase di esecuzione

Nella fase di preparazione, l'algoritmo prepara i dati necessari per l'esecuzione dell'algoritmo stesso; nella fase di esecuzione, l'algoritmo utilizza i dati preparati nella fase precedente per prendere una decisione basata sulla probabilità.

Uno degli algoritmi probabilistici più conosciuti è l'algoritmo di **Monte Carlo**. Questo algoritmo utilizza una serie di numeri casuali per simulare il comportamento di un sistema complesso. L'algoritmo di Monte Carlo è utilizzato in molti campi, tra cui la finanza, l'ingegneria, la fisica e molti altri.

Il funzionamento dell'algoritmo di Monte Carlo si basa sulla generazione di numeri casuali che vengono utilizzati per simulare il comportamento del sistema. Questi numeri casuali vengono generati da un **generatore di numeri casuali**, che produce sequenze di numeri che soddisfano determinate proprietà statistiche.

In generale, il funzionamento dell'algoritmo di Monte Carlo può essere suddiviso in tre fasi principali: generazione dei numeri casuali, simulazione del sistema e calcolo della soluzione approssimata.

1. La prima fase consiste nella **generazione di una serie di numeri casuali**. Questi numeri casuali sono generati da un generatore di numeri casuali, che produce sequenze di numeri che

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

soddisfano determinate proprietà statistiche. Questi numeri casuali sono utilizzati per simulare il comportamento del sistema.

2. Nella seconda fase, l'algoritmo utilizza i numeri casuali generati nella fase precedente per **simulare il comportamento del sistema**. Ad esempio, se si vuole simulare il comportamento di una particella in un campo magnetico, si possono generare numeri casuali per rappresentare la posizione e la velocità della particella in ogni istante di tempo. Utilizzando questi numeri casuali, l'algoritmo può simulare il comportamento della particella nel campo magnetico.
3. Nella terza fase, l'algoritmo utilizza i dati raccolti nella fase di simulazione per **calcolare una soluzione approssimata al problema**. Ad esempio, se si vuole calcolare l'area di una figura complessa, l'algoritmo può generare numeri casuali per rappresentare i punti all'interno della figura e utilizzare la proporzione tra i punti all'interno della figura e il numero totale di punti generati per stimare l'area della figura.

Come si intuisce, la principale limitazione dell'algoritmo di Monte Carlo è che richiede la generazione di un grande numero di numeri casuali per ottenere una stima accurata; inoltre, l'algoritmo può essere influenzato dalla scelta del generatore di numeri casuali utilizzato.

L'algoritmo di Monte Carlo ha molte applicazioni concrete in diversi campi:

- **Finanza**: l'algoritmo è ampiamente utilizzato nel settore finanziario per valutare le opzioni e i derivati; ad esempio, per valutare un'opzione di acquisto su un'azione, si utilizza l'algoritmo per simulare il comportamento del prezzo dell'azione nel tempo e valutare l'opzione in base ai risultati della simulazione.
- **Fisica**: l'algoritmo è utilizzato in fisica per simulare il comportamento dei sistemi fisici; ad esempio, per simulare il comportamento di un sistema di particelle, si utilizza l'algoritmo per generare numeri casuali che rappresentano la posizione e la velocità delle particelle in ogni istante di tempo.
- **Biologia**: l'algoritmo viene utilizzato in biologia per simulare il comportamento dei sistemi biologici; ad esempio, per simulare il comportamento di una proteina, si utilizza l'algoritmo per generare numeri casuali che rappresentano la posizione e l'orientamento della proteina in ogni istante di tempo.
- **Chimica**: l'algoritmo viene utilizzato in chimica per simulare il comportamento dei sistemi chimici; ad esempio, per simulare il comportamento di una reazione chimica, si utilizza l'algoritmo per generare numeri casuali che rappresentano la posizione e la velocità delle particelle chimiche in ogni istante di tempo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **Ingegneria**: l'algoritmo viene utilizzato in ingegneria per valutare la sicurezza e l'affidabilità dei sistemi; ad esempio, per valutare la resistenza di un ponte, si utilizza l'algoritmo per simulare il comportamento delle forze che agiscono sul ponte in diverse condizioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Espressione Polinomiale

L'espressione polinomiale nulla è un problema classico della matematica che si pone il seguente quesito: data una funzione polinomiale $f(x)$ di grado n, è possibile trovare n numeri reali $x_1, x_2, \ldots, x_n$ tali che $f(x_1) = f(x_2) = \ldots = f(x_n) = 0$?

L'algoritmo di Monte Carlo può essere utilizzato per risolvere questo problema in modo efficiente e probabilistico:

1. Si genera un numero casuale x compreso tra $-M$ *e* M, dove M è un valore sufficientemente grande e positivo
2. Si calcola il valore della funzione polinomiale $f(x)$. Se $f(x) = 0$ allora x è una radice dell'equazione $f(x) = 0$ e l'algoritmo termina.
3. Se $f(x) \neq 0$ si ripete il passaggio 1 e 2 un grande numero di volte.
4. Se non viene trovato alcun x tale che $f(x) = 0$, allora si conclude che $f(x)$ non ha radici reali.

L'algoritmo di Monte Carlo per l'espressione polinomiale nulla è probabilistico, il che significa che può fornire una risposta sbagliata in modo occasionale. Tuttavia, aumentando il numero di iterazioni, è possibile ridurre il rischio di errore a livelli molto bassi.

Python offre diverse librerie per l'implementazione dell'algoritmo di Monte Carlo per l'espressione polinomiale nulla, ad esempio, la libreria numpy offre la funzione numpy.roots che può essere utilizzata per trovare le radici di un polinomio:

```
import numpy as np

# Coefficienti del polinomio
coeff = [1, 2, 1, 3, 5, 7]

# Numero di iterazioni
n_iter = 10000

# Intervallo di ricerca delle radici
M = 1000

# Effettua n_iter iterazioni dell'algoritmo di Monte Carlo
for i in range(n_iter):
    # Genera un numero casuale compreso tra -M e M
    x = np.random.uniform(-M, M)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    # Calcola il valore del polinomio in x
    y = np.polyval(coeff, x)

    # Se il valore è molto vicino a zero, allora x è una radice
    if abs(y) < 1e-6:
        print("Radice trovata: x =", x)
        break

# Se nessuna radice è stata trovata, stampa un messaggio di avviso
else:
    print("Nessuna radice trovata")
```

Viene generato un numero casuale nell'intervallo $[-M, M]$, il valore del polinomio viene calcolato in quel punto, e se il valore è molto vicino a zero (entro la soglia di $1e^{-6}$), allora viene considerato un'approssimazione di una radice del polinomio.

Il ciclo for viene ripetuto n_iter volte, ognuna con una nuova iterazione dell'algoritmo di Monte Carlo:

- se viene trovata una radice durante le iterazioni, l'algoritmo si interrompe e viene stampato un messaggio che riporta il valore di x che approssima la radice
- se nessuna radice viene trovata entro le n_iter iterazioni, viene stampato un messaggio di avviso.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Bloom Filter

Un altro esempio interessante di applicazione è quello relativo ai **Bloom Filter**.

I Bloom filter sono una struttura dati probabilistica utilizzata per effettuare ricerche di dati in un insieme di dati grande, in modo efficiente e con una quantità minima di spazio.

I Bloom filter sono una combinazione di due strutture dati: i **BitSet** e le **tabelle hash**:

- il BitSet è una struttura dati che rappresenta un insieme di bit
- le tabelle hash sono strutture dati che mappano le chiavi ai valori in modo efficiente

Analizziamo da vicino la struttura di un BitSet.

Un BitSet è una struttura dati che rappresenta un insieme di bit: in un BitSet, ogni bit può essere impostato a 0 o 1, a seconda che l'elemento corrispondente sia presente o meno nell'insieme.

Il BitSet è utile quando si deve manipolare un grande insieme di dati binari, in quanto consente di eseguire operazioni booleane come l'AND, l'OR e il NOT in modo efficiente.

In Python, il BitSet può essere implementato utilizzando il tipo di dato set e le operazioni bit a bit. Ad esempio, il seguente codice crea un BitSet con 10 bit e imposta il terzo e il sesto bit a 1:

```
bitset = set(range(10))
bitset.remove(2)
bitset.remove(5)
```

In questo esempio, il BitSet viene creato utilizzando la funzione "set" di Python. Il parametro range(10) indica che il BitSet deve contenere 10 bit. Successivamente, i bit corrispondenti al terzo e al sesto elemento dell'insieme vengono impostati a 1 utilizzando la funzione "remove".

Per eseguire operazioni booleane sui BitSet, è possibile utilizzare gli operatori bit a bit di Python. Ad esempio, il seguente codice esegue l'AND tra due BitSet:

```
a = set([0, 1, 2, 3, 4])
b = set([2, 3, 4, 5, 6])
result = a & b
```

In questo esempio, l'operatore $\&$ viene utilizzato per eseguire l'AND tra i due BitSet a e b: il risultato è un nuovo BitSet che contiene solo gli elementi che sono presenti in entrambi gli insiemi. In questo caso, il risultato sarà il BitSet $\{2, 3, 4\}$.

Un Bloom Filter che utilizza BitSet funziona in questo modo:

1. Si inizializza un BitSet di n bit a 0

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Si selezionano k funzioni di hash indipendenti casuali: ogni funzione di hash mappa una chiave di input a un numero intero tra 0 e $n-1$
3. Per ogni elemento da inserire nell'insieme, si calcolano i k valori hash corrispondenti alle k funzioni di hash; si impostano i bit del BitSet corrispondenti ai k valori hash a 1
4. Per cercare un elemento nell'insieme, si calcolano i k valori hash corrispondenti alle k funzioni di hash; si controlla se tutti i bit del BitSet corrispondenti ai k valori hash sono impostati a 1:
    a. se tutti i bit sono impostati a 1, allora l'elemento potrebbe essere presente nell'insieme
    b. in caso contrario, l'elemento non è presente nell'insieme

Il vantaggio del Bloom filter è che può fornire una risposta rapida e probabilistica alla domanda "l'elemento è presente nell'insieme?". Inoltre, il Bloom filter richiede solo una quantità limitata di spazio per memorizzare l'insieme di dati, rendendolo ideale per le applicazioni che gestiscono grandi quantità di dati.

Tuttavia, l'uso di funzioni di hash casuali introduce una possibilità di falsi positivi, cioè l'indicazione che un elemento è presente nell'insieme quando in realtà non lo è.

La probabilità di un falso positivo dipende dalla dimensione del BitSet e dal numero di funzioni di hash utilizzate: aumentando la dimensione del BitSet e il numero di funzioni di hash utilizzate, è possibile ridurre la probabilità di falsi positivi.

Python offre diverse librerie per l'implementazione di Bloom filter:

```
import bitarray
import mmh3

class BloomFilter:
    def __init__(self, n, k):
        self.n = n # Numero di bit nel BitSet
        self.k = k # Numero di funzioni di hash da utilizzare
        self.bitset = bitarray.bitarray(n) # BitSet inizializzato a 0
        self.bitset.setall(False) # Imposta tutti i bit a 0

    def add(self, item):
        # Calcola i k valori di hash e imposta i bit corrispondenti a 1
        for i in range(self.k):
            hash_val = mmh3.hash(item, i) % self.n
            self.bitset[hash_val] = True

    def contains(self, item):
        # Verifica se tutti i bit corrispondenti ai k valori
        # di hash sono impostati a 1
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
        for i in range(self.k):
            hash_val = mmh3.hash(item, i) % self.n
            if not self.bitset[hash_val]:
                return False
        return True
```

In questo esempio, la classe "BloomFilter" rappresenta un Bloom filter. Il costruttore della classe prende in input il numero di bit n e il numero di funzioni di hash k da utilizzare.

Il BitSet viene inizializzato a 0 utilizzando la libreria bitarray.

Il metodo add viene utilizzato per aggiungere un elemento all'insieme di dati; per ogni elemento, vengono calcolati i k valori di hash utilizzando la funzione di hash mmh3.hash e i bit corrispondenti ai valori di hash vengono impostati a 1.

Il metodo contains viene utilizzato per verificare se un elemento è presente nell'insieme di dati; per ogni elemento, vengono calcolati i k valori di hash utilizzando la funzione di hash mmh3.hash e si verifica se tutti i bit corrispondenti ai valori di hash sono impostati a 1. Se tutti i bit sono impostati a 1, allora l'elemento potrebbe essere presente nell'insieme.

Per utilizzare il Bloom filter, si può creare un oggetto BloomFilter con il numero di bit desiderato e il numero di funzioni di hash, aggiungere gli elementi all'insieme utilizzando il metodo add e cercare gli elementi utilizzando il metodo contains.

L'implementazione utilizzando la tabella hash prevede di inizializzare una tabella hash vuota; si selezionano k funzioni di hash casuali, ognuna delle quali mappa una chiave di input a un numero intero.

Per ogni elemento da inserire nell'insieme, si calcolano i k valori hash corrispondenti alle k funzioni di hash e si impostano i bit della tabella hash corrispondenti ai k valori hash a 1.

Per cercare un elemento nell'insieme, si calcolano i k valori hash corrispondenti alle k funzioni di hash e si verifica se tutti i bit della tabella hash corrispondenti ai k valori hash sono impostati a 1. Se tutti i bit sono impostati a 1, allora l'elemento potrebbe essere presente nell'insieme.

La principale differenza tra le due implementazioni è che la versione con BitSet richiede meno spazio in memoria rispetto alla versione con tabella hash, ma può introdurre un maggiore tasso di falsi positivi; al contrario, la versione con tabella hash richiede più spazio in memoria rispetto alla versione con BitSet, ma può ridurre il tasso di falsi positivi.

In generale:

- l'implementazione con BitSet è più efficiente dal punto di vista della memoria, ma potrebbe richiedere un maggiore numero di funzioni di hash per ottenere un basso tasso di falsi positivi

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- l'implementazione con tabella hash richiede più spazio in memoria, ma può fornire un tasso di falsi positivi più basso con meno funzioni di hash

Ecco un esempio di implementazione di un Bloom filter utilizzando una tabella hash in Python:

```
import mmh3

class BloomFilter:
    def __init__(self, n, k):
        self.n = n # Numero di bit nella tabella hash
        self.k = k # Numero di funzioni di hash da utilizzare
        self.table = [0] * n # Tabella hash inizializzata a 0

    def add(self, item):
        # Calcola i k valori di hash e imposta i bit corrispondenti
        # a 1 nella tabella hash
        for i in range(self.k):
            hash_val = mmh3.hash(item, i) % self.n
            self.table[hash_val] = 1

    def contains(self, item):
        # Verifica se tutti i bit corrispondenti ai k valori di
        # hash sono impostati a 1 nella tabella hash
        for i in range(self.k):
            hash_val = mmh3.hash(item, i) % self.n
            if self.table[hash_val] == 0:
                return False
        return True
```

In questa implementazione, la classe "BloomFilter" rappresenta un Bloom filter che utilizza una tabella hash. Il costruttore della classe prende in input il numero di bit n nella tabella hash e il numero di funzioni di hash k da utilizzare. La tabella hash viene inizializzata con n elementi tutti impostati a 0.

Il metodo add viene utilizzato per aggiungere un elemento all'insieme di dati. Per ogni elemento, vengono calcolati i k valori di hash utilizzando la funzione di hash mmh3.hash e i bit corrispondenti ai valori di hash nella tabella hash vengono impostati a 1.

Il metodo contains viene utilizzato per verificare se un elemento è presente nell'insieme di dati. Per ogni elemento, vengono calcolati i k valori di hash utilizzando la funzione di hash mmh3.hash e si verifica se tutti i bit corrispondenti ai valori di hash nella tabella hash sono impostati a 1. Se tutti i bit sono impostati a 1, allora l'elemento potrebbe essere presente nell'insieme.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per utilizzare il Bloom filter, si può creare un oggetto BloomFilter con il numero di bit desiderato e il numero di funzioni di hash, aggiungere gli elementi all'insieme utilizzando il metodo add e cercare gli elementi utilizzando il metodo contains.

In generale, i Bloom filter sono utilizzati in molte applicazioni dove è necessario gestire grandi quantità di dati e velocizzare le operazioni di ricerca, ad esempio:

- **Cache**: i Bloom filter vengono utilizzati nei sistemi di caching per migliorare le prestazioni di accesso ai dati. I Bloom filter vengono utilizzati per verificare rapidamente se una risorsa è presente nella cache prima di accedere alla memoria principale.
- **Filtri antispam**: i Bloom filter vengono utilizzati per filtrare i messaggi di spam. I Bloom filter vengono utilizzati per mantenere un elenco di parole chiave associate al spam e verificare se una nuova email contiene una o più di queste parole chiave.
- **Sistemi di conteggio approssimato**: i Bloom filter possono essere utilizzati per effettuare conteggi approssimati di elementi unici in un insieme di dati molto grande.
- **Controllo della presenza di virus**: i Bloom filter vengono utilizzati nei sistemi antivirus per verificare rapidamente se un file è infetto da un virus noto.
- **Sistemi di ricerca sugli indici**: i Bloom filter vengono utilizzati nei motori di ricerca per ridurre il numero di ricerche su un indice quando una parola chiave non è presente nell'indice.

Occorre tener conto del trade-off fra occupazione di memoria e probabilità di falso positivo: sia ϵ la probabilità di falso positivo: i bloom filter richiedono $1.44\ \log_2 \frac{1}{\epsilon}$ bit per elemento inserito.

Vediamo un altro esempio di utilizzo del Bloom filter per la verifica della correttezza delle password senza doverle memorizzare:

```
import hashlib
import random
import string

class PasswordFilter:
    def __init__(self, size, num_hashes):
        self.size = size
        self.num_hashes = num_hashes
        self.bits = [False] * size

    def _hash(self, value, seed):
        m = hashlib.sha256()
        m.update(str(seed).encode('utf-8'))
        m.update(value.encode('utf-8'))
        return int.from_bytes(m.digest(), byteorder='big') % self.size
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    def add_password(self, password):
        for i in range(self.num_hashes):
            seed = random.randint(0, 2**32 - 1)
            index = self._hash(password, seed)
            self.bits[index] = True

    def check_password(self, password):
        for i in range(self.num_hashes):
            seed = random.randint(0, 2**32 - 1)
            index = self._hash(password, seed)
            if not self.bits[index]:
                return False
        return True

# Esempio di utilizzo del filtro password
pf = PasswordFilter(size=1000000, num_hashes=5)

# Aggiunge alcune password al filtro
passwords = ['password', 'secret', 'qwerty', '123456']
for password in passwords:
    pf.add_password(password)

# Verifica alcune password
print(pf.check_password('password'))  # True
print(pf.check_password('incorrect'))  # False
print(pf.check_password('qwerty'))  # True
print(pf.check_password('password123'))  # False
```

In questo esempio, viene creato un filtro di Bloom con una dimensione di 1000000 bit e 5 funzioni di hash. Viene poi aggiunta una lista di password al filtro.

Per ognuna di queste viene calcolato l'hash 5 volte utilizzando la stessa funzione hash ma con diversi valori di "seed" (Il valore di "seed" viene utilizzato per modificare il risultato della funzione di hash in modo da produrre un risultato differente per ogni "seed" diverso).

Il processo di hash consiste nel trasformare un input di lunghezza arbitraria in un output di lunghezza fissa, usando una funzione di hash specifica; in altre parole, una funzione di hash prende in input una stringa (ad esempio, una password) e produce in output un valore numerico di lunghezza fissa.

In questo modo, per ogni password viene generato un set di indici corrispondenti ai bit del filtro di Bloom; ad esempio, per una password "password123", le 5 funzioni di hash potrebbero produrre i seguenti 5 indici: 123, 456, 789, 321, 654.

Ogni volta che una nuova password viene aggiunta al Bloom filter, viene generato un'insieme di indici in questo modo e impostati a 1 nel filtro di Bloom.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Successivamente, viene verificato se alcune password sono presenti nel filtro: per fare ciò, si applicano nuovamente le stesse funzioni di hash alla password e si verificano i bit corrispondenti nel filtro di Bloom; se tutti i bit corrispondenti sono impostati a 1, la password è considerata presente nel Bloom filter (anche se potrebbe essere un falso positivo), altrimenti la password è sicuramente assente nel Bloom filter.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Alan Bertossi, Alberto Motresor: Algoritmi e strutture di dati, Città Studi Edizioni, terza edizione;
- C. Demetrescu, I. Finocchi, G. F. Italiano: Algoritmi e strutture dati, McGraw-Hill, seconda edizione;
- Crescenzi, Gambosi, Grossi: Strutture di Dati e Algoritmi, Pearson/Addison-Wesley;
- Sedgewick: Algoritmi in C, Pearson, 2015;
- Cormen Leiserson Rivest Stein-Introduzione Agli Algoritmi E Strutture Dati-Prima Edizione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).